

RECEP TAYYIP ERDOGAN UNIVERSITY

Faculty of Engineering and Architecture
Department of Computer Engineering

EMBEDDED SYSTEMS

SPLIT-SCREEN SPACE SHOOTER

Student Name:	Yakup Eroglu
Student Number:	221401045
Course:	Embedded Systems
Platform:	ESP32-S3 (PlatformIO) + Python/Pygame
Date:	June 2026

TABLE OF CONTENTS

1 Project Overview

2 System Architecture

3 Hardware Components

3.1 ESP32-S3-DevKitC-1 (N16R8)

3.2 Deneyap Joystick Module (x2)

3.3 Host PC

4 Circuit Connections

5 Communication Protocol

6 ESP32 Firmware (PlatformIO)

7 Game Software Architecture

8 Input Abstraction Layer

9 Gameplay Mechanics

10 Rendering and Resolution Scaling

11 Function and Module Reference

12 Settings and Calibration

13 Development Environment

14 Testing and Validation

15 Known Limitations

16 Future Development

1. PROJECT OVERVIEW

This project is a two-player, split-screen space shooter game. The game runs on a host PC and is written in Python using the Pygame library. Each player controls a spaceship at the bottom of their half of the screen, destroying meteors and enemy ships that descend from the top. The two players are controlled by two independent analog joysticks read by an ESP32-S3 microcontroller, which streams their state to the PC over a USB serial link.

The embedded side (ESP32-S3 firmware) and the game side (Python) are fully decoupled: the game talks only to an abstract input layer, so the same game runs identically with a keyboard or with the hardware joysticks -- only the input source changes.

Objectives

- Read two 2-axis analog joysticks + buttons on an ESP32-S3 and stream them over USB serial.
- Design a clean input-abstraction layer so keyboard and serial sources are interchangeable.
- Build a complete, real-time 2-player split-screen arcade game in Python/Pygame.
- Implement delta-time, frame-rate-independent game logic with no busy-waiting.
- Provide a headless self-test so the full game logic is verifiable without a display.
- Keep all tunable values in a single configuration module for easy balancing.

System Flow

```

2x Joystick (analog X/Y + button)
    |
    v
ESP32-S3 firmware -- reads ADC + buttons, ~66 Hz
    | "x1,y1,b1,x2,y2,b2\n" @ 115200 baud (USB serial)
    v
SerialJoystickInputSource (Python) -- parse -> InputState
    |
    v
Game loop (60 FPS): update(dt) -> collisions -> render -> flip
  
```

2. SYSTEM ARCHITECTURE

The system has two physically separate halves connected by one USB cable. The ESP32-S3 is the data-acquisition device: it only reads sensors and transmits. The PC is the simulation and rendering device: it parses incoming packets, runs all game logic, and draws the screen. No game logic runs on the ESP32, which keeps the firmware tiny and the protocol simple.

Stage	Component	Responsibility
1 Input	2x Deneyap joystick	Analog X/Y + push button per player
2 Acquire	ESP32-S3 firmware	analogRead (12-bit) + digitalRead, format line
3 Transport	USB serial @115200	Line "x1,y1,b1,x2,y2,b2" ~66 Hz
4 Decode	SerialJoystickInputSource	Parse, map to -1..1, per-player invert
5 Logic	Game / Arena	Movement, collisions, scoring, difficulty
6 Output	Pygame renderer	Fullscreen split-screen draw @60 FPS

Because the boundary between halves is the abstract `InputState` record, the keyboard path (`KeyboardInputSource`) and the hardware path (`SerialJoystickInputSource`) are drop-in replacements for one another; the game core never

changes.

3. HARDWARE COMPONENTS

3.1 ESP32-S3-DevKitC-1 (N16R8)

The central controller is an ESP32-S3 development board. It reads four analog axes and two buttons and streams them over its USB serial port. ADC1 channels (GPIO1-GPIO10) are used for the analog axes because ADC2 is shared with the radio.

Specification	Value
Processor	Xtensa LX7 dual-core, up to 240 MHz
RAM	512 KB SRAM + 8 MB PSRAM (R8)
Flash	16 MB (N16)
ADC	12-bit; ADC1 on GPIO1-10 (0..4095)
USB	Dual USB-C: native USB-CDC/JTAG + CH343 UART bridge
Strapping pins (avoid)	GPIO0, GPIO3, GPIO45, GPIO46
Framework	Arduino (via PlatformIO espressif32)

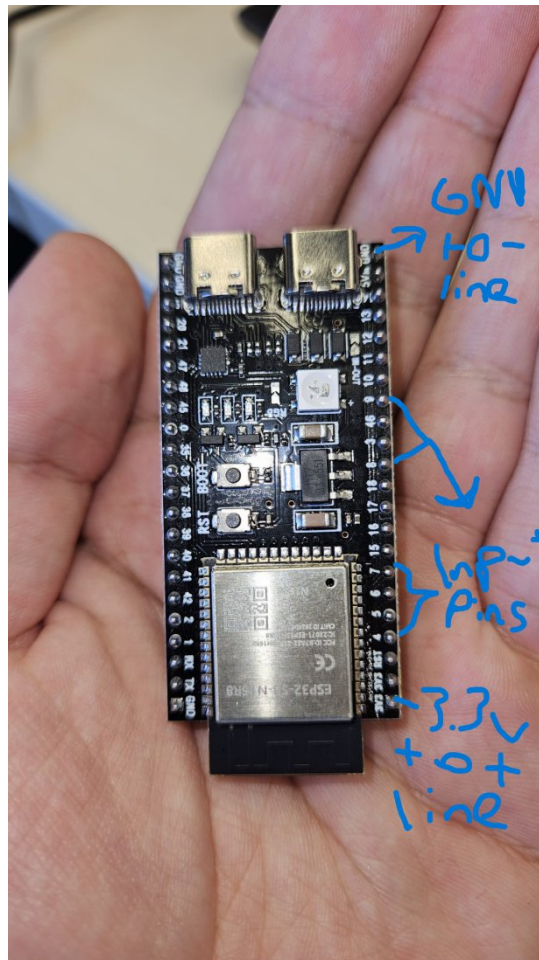


Figure 1. ESP32-S3 board -- GND to the (-) rail, signal/input pins, and 3.3V to the (+) rail.

3.2 Deneyap Joystick Module ("Kumanda Kolu", x2)

A 2-axis analog thumb joystick with an integrated push button (based on an STM8S003F3). It operates at 3.3V. The left header exposes 3V3, RES, X, Y, SW; the right (I2C) header exposes GND, SDA, SCL, SWIM, NC. This project uses the module in pure analog mode: 3V3, GND, X, Y, SW (I2C unused). On this module the SW pin reads HIGH when the knob is pressed (verified by test).

Pin	Used	Function
3V3	Yes	Power (3.3V) -- to ESP 3V3 rail
GND (I2C side)	Yes	Common ground -- to ESP GND rail
X	Yes	Horizontal analog output (0..4095)
Y	Yes	Vertical analog output (0..4095)
SW	Yes	Push button (HIGH when pressed)
RES / SDA / SCL / SWIM / NC	No	Reset / I2C / debug -- left unconnected

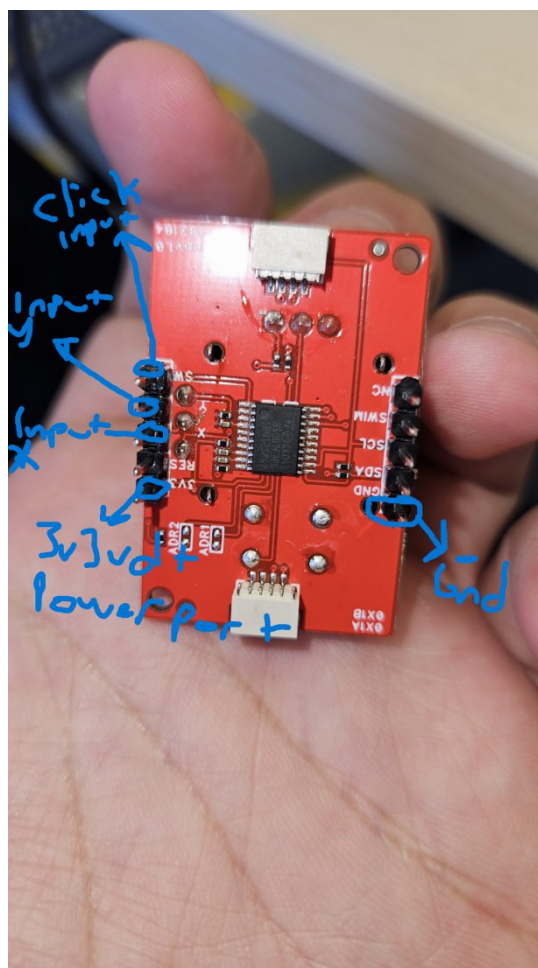


Figure 2. Deneyap joystick (back) -- SW = click input, X / Y = analog input, 3V3 = power port, GND.

3.3 Host PC

Any Windows PC running Python 3.10 and Pygame 2.5. The PC opens the joystick serial port (e.g. COM11 via the CH343 bridge), runs the game at 60 FPS, and renders fullscreen at the native desktop resolution (the process is made DPI-aware so the window fills the physical screen).

4. CIRCUIT CONNECTIONS

All six signal wires use pins GPIO4-GPIO9, which sit on the same long side of the board and are all ADC1-capable. This avoids the narrow-side pins (GPIO1/2), the strapping pins, and the USB pins (GPIO19/20). Power is shared on the breadboard rails.

Module	Module Pin	ESP32-S3 GPIO	Mode	Description
Joystick 1 (P1)	X	GPIO4	ADC1	Horizontal analog
Joystick 1 (P1)	Y	GPIO5	ADC1	Vertical analog
Joystick 1 (P1)	SW	GPIO6	INPUT_PULLUP	Button (special)
Joystick 2 (P2)	X	GPIO7	ADC1	Horizontal analog
Joystick 2 (P2)	Y	GPIO8	ADC1	Vertical analog
Joystick 2 (P2)	SW	GPIO9	INPUT_PULLUP	Button (special)
Both	3V3	3V3	Power	3.3V supply (rail)
Both	GND	GND	Ground	Common ground (rail)

ESP32-S3-DevKitC-1

```

+-----+
| GPIO4  [ADC1]  ---- Joystick1 X    |
| GPIO5  [ADC1]  ---- Joystick1 Y    |
| GPIO6  [PULLUP] -- Joystick1 SW    |
| GPIO7  [ADC1]  ---- Joystick2 X    |
| GPIO8  [ADC1]  ---- Joystick2 Y    |
| GPIO9  [PULLUP] -- Joystick2 SW    |
| 3V3    ----- both 3V3 (rail)    |
| GND    ----- both GND (rail)    |
+-----+
```

Safety: the joysticks must be powered from 3V3, NOT 5V -- the ESP32 ADC tolerates only up to ~3.3V. Signal wires are tapped from a free breadboard hole in the pin row; the +/- power rails carry power only and must never receive a signal wire.

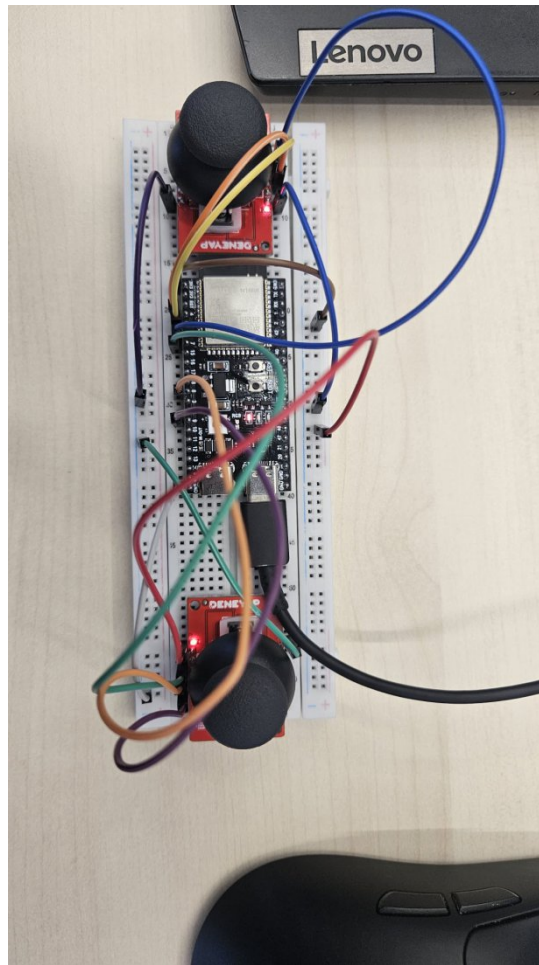


Figure 3. Assembled prototype -- both joysticks wired to the ESP32-S3 on a breadboard.

5. COMMUNICATION PROTOCOL

The ESP32 transmits one ASCII line per update, terminated by a newline. The line contains six comma-separated integers: both axes and the button for each of the two players.

Format: $x_1, y_1, b_1, x_2, y_2, b_2 \backslash n$
 Example: 2048, 3900, 1, 2050, 150, 0

x, y : raw 12-bit ADC value, 0..4095 (centre ~2048)
 b : button, 1 = pressed, 0 = released

Field	Range	Meaning
x_1 / y_1	0..4095	Player 1 joystick axes (raw ADC)
b_1	0 / 1	Player 1 button (1 = pressed)
x_2 / y_2	0..4095	Player 2 joystick axes (raw ADC)
b_2	0 / 1	Player 2 button (1 = pressed)

Baud rate is 115200; the firmware sends a packet roughly every 15 ms (~66 packets/second), comfortably above the 60 FPS game loop. On the PC, the raw value is mapped to a normalized axis value in the range -1..1, a small dead-zone removes centre jitter, and a per-player invert flag corrects mounting orientation.

6. ESP32 FIRMWARE (PLATFORMIO)

The firmware is a small PlatformIO/Arduino sketch in `firmware/src/main.cpp`. It configures the ADC and button pins once, then loops: read axes, read buttons, print the line.

Constant	Value	Role
P1_X / P1_Y	GPIO4 / GPIO5	Player 1 analog axes
P1_SW	GPIO6	Player 1 button (INPUT_PULLUP)
P2_X / P2_Y	GPIO7 / GPIO8	Player 2 analog axes
P2_SW	GPIO9	Player 2 button (INPUT_PULLUP)

```
void setup() {
    Serial.begin(115200);
    analogReadResolution(12);          // 0..4095
    analogSetAttenuation(ADC_11db);    // full ~0..3.3V range
    pinMode(P1_SW, INPUT_PULLUP);
    pinMode(P2_SW, INPUT_PULLUP);
}

void loop() {
    int x1 = analogRead(P1_X), y1 = analogRead(P1_Y);
    int x2 = analogRead(P2_X), y2 = analogRead(P2_Y);
    // This module reads HIGH when pressed:
    int b1 = (digitalRead(P1_SW) == HIGH) ? 1 : 0;
    int b2 = (digitalRead(P2_SW) == HIGH) ? 1 : 0;
    Serial.printf("%d,%d,%d,%d,%d,%d\n", x1, y1, b1, x2, y2, b2);
    delay(15);                          // ~66 packets/second
}
```

Serial is routed to the CH343 USB-UART bridge (the COM port the PC opens) via the build flag `ARDUINO_USB_CDC_ON_BOOT=0` in `platformio.ini`. The board target is `esp32-s3-devkitc-1`.

7. GAME SOFTWARE ARCHITECTURE

The Python game is organized into four layers with a strictly one-directional dependency chain. Lower layers never import higher ones, which keeps the design clean and testable.

```
settings.py    <--  input_source.py / entities.py    <--  main.py
(constants)      (input + game objects)          (game loop)
```

Module	Role
settings.py	Single source of truth for every tunable constant
input_source.py	InputState + InputSource (Keyboard / Serial / Scripted)
entities.py	Ship, Bullet, Meteor, Enemy, EnemyLaser, Particle + sprites
main.py	Game, Arena, Difficulty, state machine, render/HUD, self-test

8. INPUT ABSTRACTION LAYER

The game core only ever sees an `InputState` value object per player. An `InputSource` subclass produces these every frame, so the source of input can be swapped without touching game logic.


```
@dataclass
class InputState:
    move_x: float = 0.0 # -1..1 (analog ready)
    move_y: float = 0.0 # -1..1
    fire: bool = False # button -> special power trigger

class InputSource(ABC):
    def poll(self, events, keys) -> dict: # {1: InputState, 2: InputState}
        ...
```

Source	Used for	Notes
KeyboardInputSource	Play without hardware	WASD / arrows; -1/0/1 axes
SerialJoystickInputSource	Play with ESP32 joysticks	Parses serial, analog -1..1
ScriptedInputSource	Headless self-test	Deterministic synthetic input

Because the two joysticks can be mounted in different orientations, axis inversion is configured per player (P1_INVERT_X/Y, P2_INVERT_X/Y). A dead-zone (ADC_DEADZONE) near the centre is treated as zero so the ship never drifts on its own.

9. GAMEPLAY MECHANICS

Split-Screen and Arenas

A vertical divider splits the screen in two. The left half is Player 1, the right half is Player 2. Each half is an independent Arena with its own ship, meteors, enemies, score and health; drawing is clipped to each arena so nothing spills across.

Movement and Automatic Fire

Each ship moves in two axes within its arena. Firing is automatic at a fixed cooldown (FIRE_COOLDOWN = 0.28 s), so the button is free to trigger the special power instead.

Meteors and Difficulty Scaling

Meteors spawn at the top of each arena and fall. A global, capped difficulty rises over time: spawns get more frequent and meteors fall faster, up to a ceiling so the game never becomes impossible.

Difficulty parameter	Start	Cap
Meteor spawn interval	1.30 s	0.35 s
Fall-speed multiplier	1.0x	2.6x
Time to reach the cap	--	90 s

Enemy Ships and Lasers

Occasionally (rarer than meteors) an enemy ship spawns in BOTH arenas at the same time, which keeps the two players fair. Enemy ships patrol horizontally in a band near the top (they never descend) and periodically fire a laser aimed at the targeted ship's position at the moment of firing. A laser that hits the player deals fixed damage; an enemy ship is destroyed after three hits and is worth bonus score.

Enemy parameter	Value
Spawn interval (global, both arenas)	6 - 11 s
Hit points (bullets to destroy)	3

Enemy parameter	Value
Score on destroy	60
Laser damage	15 HP (fixed)
Max enemies per arena	2

Special Power

Every 15 asteroids or enemies destroyed by gunfire charges a player's special power (shown as a meter in the HUD). When ready, pressing the button clears every threat in that player's arena (meteors, enemies, lasers) with an explosion flash. The special awards no score, so it is a rescue tool, not a points farm.

Health (HP) and Damage

Each player starts with 100 HP. Meteors deal damage by size, lasers deal a fixed amount, and crashing into an enemy hull is heaviest. A short invulnerability window after each hit prevents overlapping collisions from draining all health at once. When HP reaches 0 that side is eliminated.

Damage source	HP lost
Small meteor	8
Medium meteor	15
Large meteor	25
Enemy laser	15
Enemy hull crash	30
Invulnerability window	0.7 s

Scoring and Game Over

Destroying a meteor scores by size (10 / 20 / 40 for small / medium / large); an enemy scores 60. Scores appear top-left (P1) and top-right (P2). When both players are eliminated, a full-screen game-over screen shows the winner (higher score; tie if equal). Press R to restart, ESC to quit.

10. RENDERING AND RESOLUTION SCALING

The game opens fullscreen at the native desktop resolution. On Windows the process is made DPI-aware and the display is opened with `set_mode((0,0), FULLSCREEN)`, so the window fills the physical screen even when Windows display scaling is enabled. A global `SCALE = height / 1080` makes all sizes, speeds and fonts adapt to any resolution while preserving the same feel.

Player ships are drawn from PNG sprites in `assets/` (`ship_p1.png` red, `ship_p2.png` purple), produced from Aseprite source files. If the sprite files are missing, the ship falls back to a drawn triangle so the game still runs. Everything else (meteors, bullets, enemies, lasers, particles, HUD) is drawn with Pygame primitives -- no other assets are required.

Damage feedback is a red overlay on the hit player's half: an instant flash on each hit plus a persistent red tint that intensifies as HP drops. The HUD shows each player's score, a colour-coded health value (green/yellow/red) and the special-power meter, plus a central difficulty level indicator.

11. FUNCTION AND MODULE REFERENCE

`Game.run()`

Main loop: events, dt = clock.tick(60), poll input, update, draw, flip.

Game.update(dt, inputs)

State machine; advances arenas, difficulty, and global enemy spawn timer.

Arena.update(dt, input_state, difficulty)

Per-player world: ship, bullets, meteors, enemies, collisions.

Arena._take_damage(amount)

Applies HP damage (respecting i-frames), triggers hurt flash, handles death.

Arena._activate_special()

Clears all threats in the arena; resets the special charge.

Ship.try_fire()

Automatic fire: returns True when the cooldown has elapsed.

Ship.draw(surf)

Blits the scaled sprite (or triangle fallback) with hit-blink.

Meteor.damage / .points

Size-based damage and score for a meteor.

Enemy.update / want_fire

Horizontal patrol; periodic laser-fire request.

SerialJoystickInputSource.poll()

Reads the serial line, maps to InputState (per-player invert).

get_ship_image(player_id)

Loads and caches the player sprite from assets/ (optional).

run_selftest()

Headless dummy-SDL run of the real game logic; exits 0 on success.

12. SETTINGS AND CALIBRATION

All gameplay values live in settings.py, so balancing requires editing only that one file. Key constants:

Constant	Value	Role
START_HEALTH	100	Starting HP per player
FIRE_COOLDOWN	0.28 s	Automatic fire interval
INVULN_TIME	0.7 s	Invulnerability after a hit
SPECIAL_KILLS_REQUIRED	15	Kills to charge the special power
SHIP_IMG_SCALE	2.6	Ship sprite size (x ship.size)
ADC_DEADZONE	0.08	Joystick centre dead-zone
SERIAL_PORT / SERIAL_BAUD	COM11 / 115200	Joystick serial port

Axis calibration is per player: if a ship moves the wrong way, flip P1_INVERT_X / P1_INVERT_Y or P2_INVERT_X / P2_INVERT_Y. The two joysticks can be mounted differently, so each player has its own invert flags. If a ship drifts at rest, increase ADC_DEADZONE.

13. DEVELOPMENT ENVIRONMENT

Software	Version	Purpose
Python	3.10.7	Game runtime
Pygame	2.5.2	Graphics, input, timing

Software	Version	Purpose
pyserial	3.5	Read the joystick serial port
PlatformIO Core	6.1.19	Build/flash the ESP32 firmware
espressif32 platform	Arduino framework	ESP32-S3 toolchain
Aseprite	--	Author the ship sprites

```
[env:esp32s3]
platform = espressif32
board = esp32-s3-devkitc-1
framework = arduino
monitor_speed = 115200
build_flags = -DARDUINO_USB_CDC_ON_BOOT=0
```

A helper, `tools/aseprite_to_png.py`, converts the Aseprite source files in `Photos/` into the `assets/` PNG sprites, so the artwork can be edited and re-exported with one command.

14. TESTING AND VALIDATION

Before running the game, `serial_test.py` prints the raw serial lines so the wiring and firmware can be checked directly (moving a joystick must change `x/y`; pressing the button must set `b` to 1).

The game also ships with a headless self-test that runs the real `update/collision/render` code with synthetic input under a dummy SDL driver, then exits with code 0 -- it can run on a machine with no display:

```
$ python main.py --selftest
[selftest] OK - frames=120, elapsed=2.00s, P1=110 P2=70, collisions_seen=True
```

The codebase was additionally hardened with several multi-agent review passes (correctness, balance, regression and tidiness), whose confirmed findings were applied and re-verified against the self-test.

15. KNOWN LIMITATIONS

- The joystick path needs `pyserial` installed and the correct COM port set in `settings.py`.
- Serial is a wired USB link; there is no wireless play yet.
- In keyboard mode both players share one keyboard.
- The game has no sound effects yet (visual feedback only).
- There is no persistent high-score storage; scores reset each run.
- Joystick axis orientation must be calibrated once per mounting.

16. FUTURE DEVELOPMENT

Short Term

- Sound effects and music (Pygame mixer).
- On-screen serial-connection status indicator.
- More meteor and enemy varieties.

Medium Term

- Power-ups (shield, rapid fire, multi-shot).
- Wireless controllers over Wi-Fi / ESP-NOW instead of USB.
- Persistent high-score table.

Long Term

- Networked play across two PCs.
- Custom PCB + 3D-printed joystick enclosure.
- Configurable controls / calibration screen inside the game.

QUICK REFERENCE CARD

Split-Screen Space Shooter -- ESP32-S3 + Dual Joystick

CONTROLS

Move P1	W A S D	left arena
Move P2	Arrow keys	right arena
Fire	AUTOMATIC	both players
Special power	SPACE / ENTER	or joystick button
Pause / Restart	P / R	ESC = quit

PIN MAP (ESP32-S3)

P1 X / Y / SW	GPIO4 / 5 / 6	ADC1
P2 X / Y / SW	GPIO7 / 8 / 9	ADC1
Power	3V3 + GND	both joysticks

RUN

Game (keyboard)	<code>python main.py</code>
Game (joystick)	<code>python main.py --serial</code>
Flash ESP	<code>pio run -d firmware -t upload</code>

TURKCE DOKUMANTASYON

Bolunmus Ekran Uzay Oyunu -- Proje Raporu



RECEP TAYYIP ERDOGAN UNIVERSITY

Muhendislik ve Mimarlik Fakultesi
Bilgisayar Muhendisligi Bolumu

GOMULU SISTEMLER

BOLUNMUS EKCRAN UZAY OYUNU

Ogrenci Adi:	Yakup Eroglu
Ogrenci Numarasi:	221401045
Ders:	Gomulu Sistemler
Platform:	ESP32-S3 (PlatformIO) + Python/Pygame
Tarih:	Haziran 2026

ICERIK TABLOSU

1 Projeye Genel Bakis

2 Sistem Mimarisi

3 Kullanilan Donanim

3.1 ESP32-S3-DevKitC-1 (N16R8)

3.2 Deneyap Joystick Modulu (x2)

3.3 Ana Bilgisayar

4 Devre Baglantilari

5 Haberlesme Protokolu

6 ESP32 Firmware (PlatformIO)

7 Oyun Yazilim Mimarisi

8 Giris Soyutlama Katmani

9 Oyun Mekanikleri

10 Render ve Cozunurluk Olcekleme

11 Fonksiyon ve Modul Referansi

12 Ayarlar ve Kalibrasyon

13 Gelistirme Ortami

14 Test ve Dogrulama

15 Bilinen Kisitlamalar

16 Gelecek Gelistirme Fikirleri

1. PROJEYE GENEL BAKIS

Bu proje, 2 oyunculu, bolunmus ekran bir uzay atari oyunudur. Oyun bir ana bilgisayarda calisir ve Python ile Pygame kutuphanesi kullanilarak yazilmistir. Her oyuncu, ekranin kendi yarisinin altindaki bir uzay aracini kontrol eder ve yukaridan inen meteorlari ve rakip gemileri yok eder. Iki oyuncu, bir ESP32-S3 mikrodenetleyici tarafindan okunan iki bagimsiz analog joystick ile kontrol edilir; ESP32 bu verileri USB seri hat uzerinden bilgisayara aktarir.

Gomulu taraf (ESP32-S3 firmware) ile oyun tarafi (Python) tamamen ayriktir: oyun yalnızca soyut bir giris katmaniyla konusur, böylece ayni oyun klavye ile de donanim joystickleriyle de aynen calisir -- yalnızca giris kaynagi degisir.

Hedefler

- ESP32-S3 ile iki 2-eksenli analog joystick + butonu okuyup USB seriden yollamak.
- Klavye ve seri kaynaklarin yer degistirebilecegi temiz bir giris soyutlamasi tasarlamak.
- Python/Pygame ile eksiksiz, gercek zamanli 2 oyunculu bolunmus ekran oyunu gelistirmek.
- Delta-time tabanlı, kare hizindan bagimsiz oyun mantigi (mesgul bekleme olmadan) yazmak.
- Ekransiz ortamda tum oyun mantigini dogrulayan bir self-test saglamak.
- Tum ayarlanabilir degerleri tek bir yapilandirma modulunde tutmak.

Sistem Akisi

```

2x Joystick (analog X/Y + buton)
|
v
ESP32-S3 firmware -- ADC + buton okur, ~66 Hz
| "x1,y1,b1,x2,y2,b2\n" @ 115200 baud (USB seri)
|
v
SerialJoystickInputSource (Python) -- ayristir -> InputState
|
v
Oyun dongusu (60 FPS): update(dt) -> carpisma -> render -> flip

```

2. SISTEM MIMARISI

Sistem, tek bir USB kablosuyla baglanan iki fiziksel parcadan olusur. ESP32-S3 veri-toplama cihazidir: yalnızca sensorleri okur ve gonderir. Bilgisayar ise simulasyon ve render cihazidir: gelen paketleri ayristirir, tum oyun mantigini calistirir ve ekrani cizer. ESP32 uzerinde oyun mantigi calismaz; bu sayede firmware kucuk, protokol basit kalir.

Asama	Bilesen	Sorumluluk
1 Giris	2x Deneyap joystick	Oyuncu basina analog X/Y + buton
2 Okuma	ESP32-S3 firmware	analogRead (12-bit) + digitalRead, satir formatla
3 Tasima	USB seri @115200	Satir "x1,y1,b1,x2,y2,b2" ~66 Hz
4 Cozme	SerialJoystickInputSource	Ayristir, -1..1 map, oyuncu basina invert
5 Mantik	Game / Arena	Hareket, carpisma, skor, zorluk
6 Cikis	Pygame render	Tam ekran bolunmus cizim @60 FPS

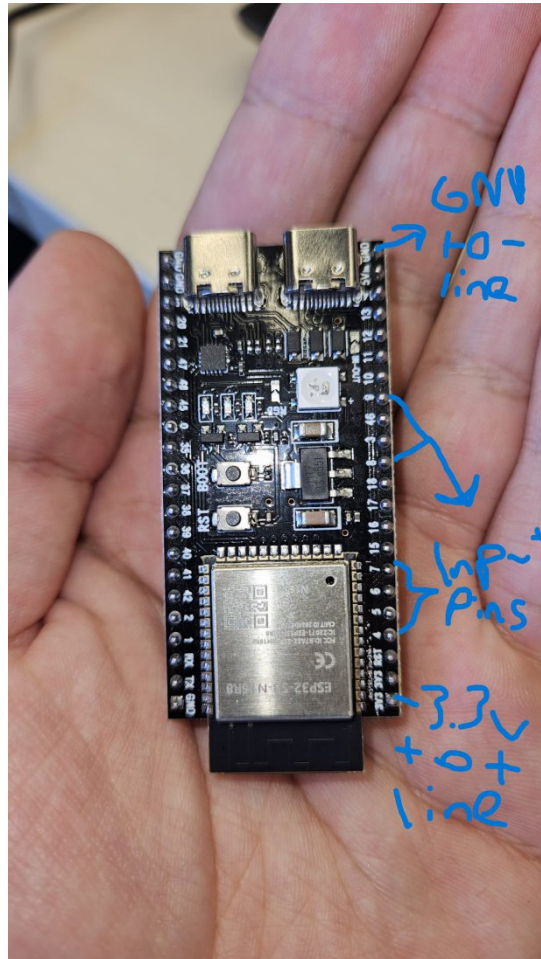
Iki parca arasindaki sinir soyut InputState kaydi oldugundan, klavye yolu (KeyboardInputSource) ve donanim yolu (SerialJoystickInputSource) birbirinin yerine tak-cikar kullanilabilir; oyun cekirdegi hic degismez.

3. KULLANILAN DONANIM

3.1 ESP32-S3-DevKitC-1 (N16R8)

Ana denetleyici bir ESP32-S3 gelistirme kartidir. Dort analog ekseni ve iki butonu okuyup USB seri portundan yollar. Analog eksenler icin ADC1 kanallari (GPIO1-GPIO10) kullanilir cunku ADC2 telsiz ile paylasilir.

Ozellik	Deger
Islemci	Xtensa LX7 cift cekirdek, 240 MHz'e kadar
RAM	512 KB SRAM + 8 MB PSRAM (R8)
Flash	16 MB (N16)
ADC	12-bit; ADC1 GPIO1-10 (0..4095)
USB	Cift USB-C: native USB-CDC/JTAG + CH343 UART kopru
Strapping pinleri (kacin)	GPIO0, GPIO3, GPIO45, GPIO46
Cerceve	Arduino (PlatformIO espressif32 ile)



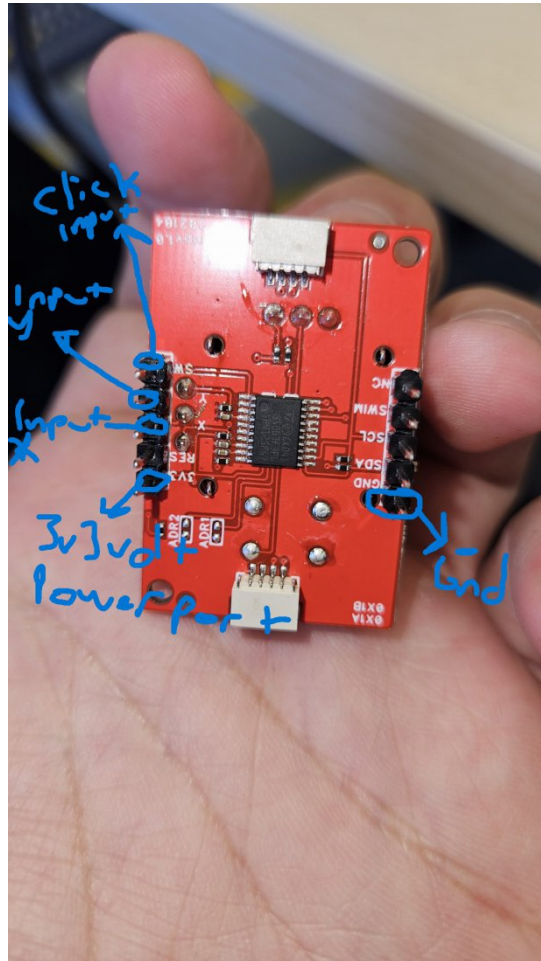
Sekil 1. ESP32-S3 karti -- GND (-) rayina, sinyalgiris pinleri ve 3.3V (+) rayina.

3.2 Deneyap Joystick Modulu ("Kumanda Kolu", x2)

2-eksenli analog joystick ve entegre basma butonu (STM8S003F3 tabanlı). 3.3V calisir. Sol header 3V3, RES, X, Y, SW; sag (I2C) header GND, SDA, SCL, SWIM, NC pinlerini sunar. Bu projede modul saf analog modda kullanilir: 3V3,

GND, X, Y, SW (I2C kullanılmaz). Bu modulde SW pini, kola basilinca HIGH okunur (test edildi).

Pin	Kullanim	Fonksiyon
3V3	Evet	Guc (3.3V) -- ESP 3V3 rayina
GND (I2C tarafi)	Evet	Ortak toprak -- ESP GND rayina
X	Evet	Yatay analog cikis (0..4095)
Y	Evet	Dikey analog cikis (0..4095)
SW	Evet	Basma butonu (basilinca HIGH)
RES / SDA / SCL / SWIM / NC	Hayir	Reset / I2C / debug -- bos birakilir



Sekil 2. Deneyap joystick (arka) -- SW = click girisi, X / Y = analog giris, 3V3 = guc portu, GND.

3.3 Ana Bilgisayar

Python 3.10 ve Pygame 2.5 calistiran herhangi bir Windows bilgisayar. PC, joystick seri portunu acar (orn. CH343 koprusu uzerinden COM11), oyunu 60 FPS'de calistirir ve native masaustu cozunurlugunde tam ekran cizer (surec DPI-aware yapildigi icin pencere fiziksel ekrani tam kaplar).

4. DEVRE BAGLANTILARI

Alti sinyal kablusunun tamamı GPIO4-GPIO9 pinlerini kullanir; bunlar kartin ayni uzun kenarinda ve hepsi ADC1 destekli. Boylece dar kenar pinleri (GPIO1/2), strapping pinleri ve USB pinleri (GPIO19/20) kullanilmaz. Gucler breadboard raylari uzerinden paylasilir.

Modul	Modul Pini	ESP32-S3 GPIO	Mod	Aciklama
Joystick 1 (P1)	X	GPIO4	ADC1	Yatay analog
Joystick 1 (P1)	Y	GPIO5	ADC1	Dikey analog
Joystick 1 (P1)	SW	GPIO6	INPUT_PULLUP	Buton (ozel guc)
Joystick 2 (P2)	X	GPIO7	ADC1	Yatay analog
Joystick 2 (P2)	Y	GPIO8	ADC1	Dikey analog
Joystick 2 (P2)	SW	GPIO9	INPUT_PULLUP	Buton (ozel guc)
Ikisi	3V3	3V3	Guc	3.3V besleme (ray)
Ikisi	GND	GND	Toprak	Ortak toprak (ray)

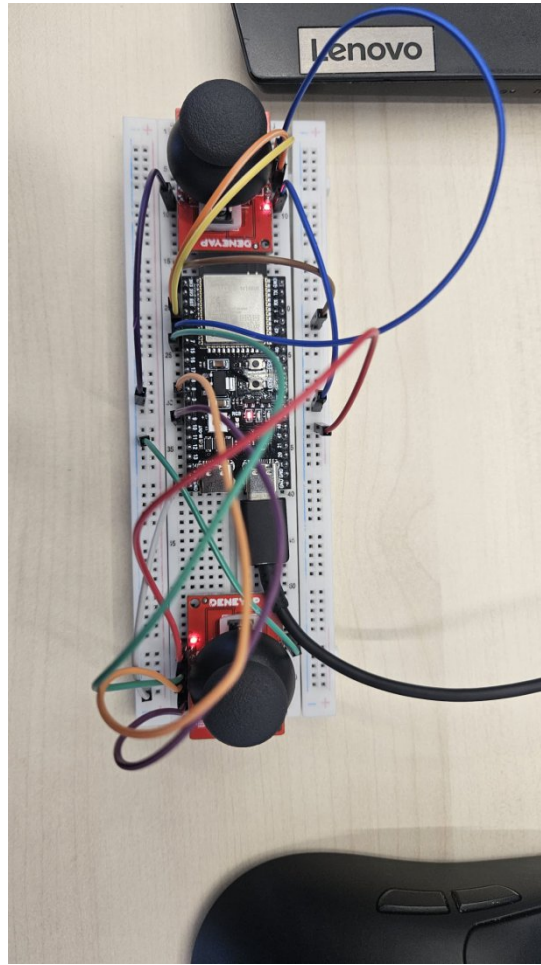
ESP32-S3-DevKitC-1

```

+-----+
| GPIO4  [ADC1]  ---- Joystick1 X    |
| GPIO5  [ADC1]  ---- Joystick1 Y    |
| GPIO6  [PULLUP] -- Joystick1 SW    |
| GPIO7  [ADC1]  ---- Joystick2 X    |
| GPIO8  [ADC1]  ---- Joystick2 Y    |
| GPIO9  [PULLUP] -- Joystick2 SW    |
| 3V3    ----- iki 3V3 (ray)       |
| GND    ----- iki GND (ray)       |
+-----+

```

Guvenlik: joystickler 3V3'ten beslenmeli, 5V'tan DEGIL -- ESP32 ADC en fazla ~3.3V tolere eder. Sinyal kablolari pin sirasindaki bos bir breadboard deliginden alinir; +/- guc raylari yalnizca guc tasir, asla sinyal kablosu baglanmaz.



Sekil 3. Kurulu prototip -- iki joystick breadboard üzerinde ESP32-S3e bagli.

5. HABERLESME PROTOKOLU

ESP32, her guncellemede satir sonu ile biten tek bir ASCII satir gonderir. Satir, virgulle ayrilmis alti tam sayi icerir: iki oyuncunun her biri icin iki eksen ve buton.

Format: x1,y1,b1,x2,y2,b2\n
Ornek: 2048,3900,1,2050,150,0

x,y : ham 12-bit ADC degeri, 0..4095 (merkez ~2048)
b : buton, 1 = basili, 0 = serbest

Alan	Aralik	Anlam
x1 / y1	0..4095	Oyuncu 1 joystick eksenleri (ham ADC)
b1	0 / 1	Oyuncu 1 butonu (1 = basili)
x2 / y2	0..4095	Oyuncu 2 joystick eksenleri (ham ADC)
b2	0 / 1	Oyuncu 2 butonu (1 = basili)

Baud hizi 115200; firmware yaklasik her 15 ms'de bir paket gonderir (~66 paket/saniye), 60 FPS oyun dongusunun rahatca uzerinde. PC tarafinda ham deger -1..1 araligina map'lenir, kucuk bir olu bolge merkez titremesini siler ve oyuncu basina invert bayragi montaj yonunu duzeltir.

6. ESP32 FIRMWARE (PLATFORMIO)

Firmware, firmware/src/main.cpp icinde kucuk bir PlatformIO/Arduino programidir. ADC ve buton pinlerini bir kez yapilandirir, sonra donguye girer: eksenleri oku, butonlari oku, satiri yazdir.

Sabit	Deger	Rol
P1_X / P1_Y	GPIO4 / GPIO5	Oyuncu 1 analog eksenleri
P1_SW	GPIO6	Oyuncu 1 butonu (INPUT_PULLUP)
P2_X / P2_Y	GPIO7 / GPIO8	Oyuncu 2 analog eksenleri
P2_SW	GPIO9	Oyuncu 2 butonu (INPUT_PULLUP)

```
void setup() {
  Serial.begin(115200);
  analogReadResolution(12); // 0..4095
  analogSetAttenuation(ADC_11db); // tam ~0..3.3V araligi
  pinMode(P1_SW, INPUT_PULLUP);
  pinMode(P2_SW, INPUT_PULLUP);
}

void loop() {
  int x1 = analogRead(P1_X), y1 = analogRead(P1_Y);
  int x2 = analogRead(P2_X), y2 = analogRead(P2_Y);
  // Bu modul basilinca HIGH okur:
  int b1 = (digitalRead(P1_SW) == HIGH) ? 1 : 0;
  int b2 = (digitalRead(P2_SW) == HIGH) ? 1 : 0;
  Serial.printf("%d,%d,%d,%d,%d,%d\n", x1,y1,b1,x2,y2,b2);
  delay(15); // ~66 paket/saniye
}
```

Serial, platformio.ini icindeki ARDUINO_USB_CDC_ON_BOOT=0 bayragi ile CH343 USB-UART koprusune (PC'nin actigi COM portu) yonlendirilir. Kart hedefi esp32-s3-devkitc-1'dir.

7. OYUN YAZILIM MIMARISI

Python oyunu, kesin olarak tek yonlu bir bagimlilik zinciri olan dort katmana ayrilmistir. Alt katmanlar ust katmanlari asla import etmez; bu da tasarimi temiz ve test edilebilir tutar.

```
settings.py <-- input_source.py / entities.py <-- main.py
(sabitler)      (giris + oyun nesneleri)      (oyun dongusu)
```

Modul	Rol
settings.py	Tum ayarlanabilir sabitlerin tek kaynagi
input_source.py	InputState + InputSource (Klavye / Seri / Scripted)
entities.py	Ship, Bullet, Meteor, Enemy, EnemyLaser, Particle + sprite
main.py	Game, Arena, Difficulty, durum makinesi, render/HUD, self-test

8. GIRIS SOYUTLAMA KATMANI

Oyun cekirdegi her zaman yalnızca oyuncu basina bir InputState deger nesnesi gorur. Bir InputSource alt sinifi bunlari her kare uretir; Boylece giris kaynagi, oyun mantigina dokunmadan degistirilebilir.


```
@dataclass
class InputState:
    move_x: float = 0.0 # -1..1 (analog hazir)
    move_y: float = 0.0 # -1..1
    fire: bool = False # buton -> ozel guc tetigi

class InputSource(ABC):
    def poll(self, events, keys) -> dict: # {1: InputState, 2: InputState}
    ...
```

Kaynak	Kullanim	Notlar
KeyboardInputSource	Donanimsiz oynama	WASD / oklar; -1/0/1 eksen
SerialJoystickInputSource	ESP32 joystick ile oynama	Seri ayristir, analog -1..1
ScriptedInputSource	Ekransiz self-test	Deterministik sentetik giris

İki joystick farklı yonlerde monte edilebildiginden, eksen tersleme oyuncu basina ayarlanir (P1_INVERT_X/Y, P2_INVERT_X/Y). Merkeze yakin bir olu bolge (ADC_DEADZONE) sifir sayilir; Boylece arac kendiliginden kaymaz.

9. OYUN MEKANIKLERI

Bolunmus Ekran ve Arenalar

Dikey bir ayirici ekrani ikiye boler. Sol yari Oyuncu 1, sag yari Oyuncu 2'dir. Her yari; kendi gemisi, meteorlari, rakipleri, skoru ve canı olan bagimsiz bir Arena'dir; cizim her arenaya kirpilir, Boylece tasma olmaz.

Hareket ve Otomatik Ates

Her gemi kendi arenasi icinde iki eksende hareket eder. Ates, sabit bir cooldown ile otomatiktir (FIRE_COOLDOWN = 0.28 sn); Boylece buton ozel gucu tetiklemek icin serbest kalir.

Meteorlar ve Zorluk Olcekleme

Meteorlar her arenanın ustunde olusup duser. Zamanla artan, tavanli bir global zorluk: dogus sikligi artar, meteorlar daha hizli duser; ancak bir tavana kadar, Boylece oyun asla imkansiz hale gelmez.

Zorluk parametresi	Baslangic	Tavan
Meteor dogus araligi	1.30 sn	0.35 sn
Dusus hizi carpani	1.0x	2.6x
Tavana ulasma suresi	--	90 sn

Rakip Gemiler ve Lazerler

Arada bir (meteorlardan nadir) bir rakip gemi, iki arenada da AYNI ANDA olusur; bu iki oyuncuyu adil tutar. Rakip gemiler ustte bir bantta yatay devriye yapar (asla asagi inmez) ve periyodik olarak, atildigi andaki hedef geminin konumuna dogru bir lazer atar. Oyuncuya carpan lazer sabit hasar verir; rakip gemi uc isabette yok olur ve bonus puan verir.

Rakip parametresi	Deger
Dogus araligi (global, iki arena)	6 - 11 sn
Can (yok etmek icin mermi)	3
Yok edince puan	60

Rakip parametresi	Deger
Lazer hasari	15 HP (sabit)
Arena basina max rakip	2

Ozel Guc

Mermiyle yok edilen her 15 asteroid veya rakip, bir oyuncunun ozel gucunu doldurur (HUD'da gosterge olarak gorulur). Hazir oldugunda butona basinca o oyuncunun arenasindaki tum tehditler (meteorlar, rakipler, lazerler) bir patlama parlamasiyla temizlenir. Ozel guc puan vermez; yani bir kurtarma aracidir, puan kaynagi degil.

Can (HP) ve Hasar

Her oyuncu 100 HP ile baslar. Meteorlar boyuta gore hasar verir, lazerler sabit hasar verir, rakip govdesine carpmak en agir olanidir. Her vurustan sonraki kisa bir dokunulmazlik penceresi, ust uste carpismalarin tum cani bir anda goturmesini onler. HP 0 olunca o taraf elenir.

Hasar kaynagi	Kaybedilen HP
Kucuk meteor	8
Orta meteor	15
Buyuk meteor	25
Rakip lazeri	15
Rakip govde carpmasi	30
Dokunulmazlik penceresi	0.7 sn

Skor ve Oyun Sonu

Meteor yok etmek boyuta gore puan verir (kucuk/orta/buyuk icin 10 / 20 / 40); rakip 60 puandir. Skorlar sol-ust (P1) ve sag-ust (P2) gosterilir. Iki oyuncu da elenince tam ekran bir oyun-sonu ekrani kazanani gosterir (yuksek skor; esitse berabere). R ile yeniden baslar, ESC ile cikilir.

10. RENDER VE COZUNURLUK OLCEKLEME

Oyun, native masaustu cozunurlugunde tam ekran acilir. Windows'ta surec DPI-aware yapilir ve ekran set_mode((0,0), FULLSCREEN) ile acilir; Boylece Windows ekran olceklemeye acik olsa bile pencere fiziksel ekrani tam kaplar. Global SCALE = yukseklik / 1080, tum boyut, hiz ve fontlarin herhangi bir cozunurluge ayni hissi koruyarak uyum saglamasini saglar.

Oyuncu gemileri assets/ icindeki PNG sprite'lariyla cizilir (ship_p1.png kirmizi, ship_p2.png mor); bunlar Aseprite kaynak dosyalarindan uretilmistir. Sprite dosyaları yoksa gemi cizilen bir ucgene duser, Boylece oyun yine calisir. Geri kalan her sey (meteor, mermi, rakip, lazer, partikul, HUD) Pygame primitifleriyle cizilir -- baska varlik gerekmez.

Hasar geri bildirimi, vurulan oyuncunun yarisinda kirmizi bir kaplamadir: her vurusta anlik bir flash ve HP dustukce siddetlenen kalici bir kirmizi tint. HUD her oyuncunun skorunu, renk kodlu can degerini (yesil/sari/kirmizi) ve ozel guc gostergesini, ayrica ortada bir zorluk seviyesi gostergesini gosterir.

11. FONKSIYON VE MODUL REFERANSI

Game.run()

Ana dongu: event, dt = clock.tick(60), giris oku, update, draw, flip.

Game.update(dt, inputs)

Durum makinesi; arenalari, zorlugu ve global rakip dogus timer'ini iletir.

Arena.update(dt, input_state, difficulty)

Oyuncu dunyasi: gemi, mermi, meteor, rakip, carpisma.

Arena._take_damage(amount)

HP hasari uygular (i-frame korur), kirmizi flash tetikler, olumu yoneter.

Arena._activate_special()

Arenadaki tum tehditleri temizler; özel guc sayacini sifirlar.

Ship.try_fire()

Otomatik ates: cooldown dolduysa True doner.

Ship.draw(surf)

Olcekli sprite'i (ya da ucgen fallback) vurur-blink ile cizer.

Meteor.damage / .points

Meteorun boyuta gore hasar ve puani.

Enemy.update / want_fire

Yatay devriye; periyodik lazer-ates istegi.

SerialJoystickInputSource.poll()

Seri satiri okur, InputState'e map'ler (oyuncu basina invert).

get_ship_image(player_id)

Oyuncu sprite'ini assets/'ten yukler + cache'ler (opsiyonel).

run_selftest()

Gercek oyun mantigini dummy-SDL ile ekransiz calistirir; basaride 0 ile cikar.

12. AYARLAR VE KALIBRASYON

Tum oyun degerleri settings.py icindedir; dengeleme yalnızca o tek dosyayi duzenlemeyi gerektirir. Temel sabitler:

Sabit	Deger	Rol
START_HEALTH	100	Oyuncu basina baslangic HP
FIRE_COOLDOWN	0.28 sn	Otomatik ates araligi
INVULN_TIME	0.7 sn	Vurus sonrasi dokunulmazlik
SPECIAL_KILLS_REQUIRED	15	Ozel gucu dolduran vurur sayisi
SHIP_IMG_SCALE	2.6	Gemi sprite boyutu (x ship.size)
ADC_DEADZONE	0.08	Joystick merkez olu bolgesi
SERIAL_PORT / SERIAL_BAUD	COM11 / 115200	Joystick seri portu

Eksen kalibrasyonu oyuncu basinadir: bir gemi ters yone gidiyorsa P1_INVERT_X / P1_INVERT_Y veya P2_INVERT_X / P2_INVERT_Y degerini cevir. Iki joystick farkli monte edilebildiginden her oyuncunun kendi invert bayraklari vardir. Bir gemi bostayken kayiyorsa ADC_DEADZONE'u buyut.

13. GELISTIRME ORTAMI

Yazilim	Surum	Amac
Python	3.10.7	Oyun calisma ortami
Pygame	2.5.2	Grafik, giris, zamanlama
pyserial	3.5	Joystick seri portunu okuma

Yazilim	Surum	Amac
PlatformIO Core	6.1.19	ESP32 firmware derleme/yukleme
espressif32 platform	Arduino cerceve	ESP32-S3 arac zinciri
Aseprite	--	Gemi sprite'larini cizme

```
[env:esp32s3]
platform = espressif32
board = esp32-s3-devkitc-1
framework = arduino
monitor_speed = 115200
build_flags = -DARDUINO_USB_CDC_ON_BOOT=0
```

Bir yardimci olan tools/aseprite_to_png.py, Photos/ icindeki Aseprite kaynak dosyalarini assets/ PNG sprite'larina ceviri; Boylece grafik tek komutla duzenlenip yeniden uretilebilir.

14. TEST VE DOGRULAMA

Oyunu calistirmadan once serial_test.py ham seri satirlari yazdirir; Boylece kablolama ve firmware dogrudan kontrol edilir (joystigi oynatinca x/y degismeli; butona basinca b 1 olmalidir).

Oyun ayrica, gercek update/carpisma/render kodunu sentetik girisle ve dummy SDL surucusuyla calistirip 0 koduyla cikan ekransiz bir self-test ile gelir -- ekransiz bir makinede de calisabilir:

```
$ python main.py --selftest
[selftest] OK - frames=120, elapsed=2.00s, P1=110 P2=70, collisions_seen=True
```

Kod tabani ayrica birkac cok-ajanli inceleme gecisi (dogruluk, denge, regresyon ve duzen) ile saglamlastirildi; onaylanan bulgular uygulandi ve self-test ile yeniden dogrulandi.

15. BILINEN KISITLAMALAR

- Joystick yolu pyserial kurulumunu ve settings.py'de dogru COM portunu gerektirir.
- Seri, kablolu bir USB baglantisidir; henuz kablosuz oynama yok.
- Klavye modunda iki oyuncu tek klavyeyi paylasir.
- Oyunun henuz ses efekti yok (yalnizca gorsel geri bildirim).
- Kalici skor saklama yok; skorlar her calistirmada sifirlanir.
- Joystick eksen yonu her montaj icin bir kez kalibre edilmelidir.

16. GELECEK GELISTIRME FIKIRLERI

Kisa Vadeli

- Ses efektleri ve muzik (Pygame mixer).
- Ekranda seri-baglanti durum gostergesi.
- Daha cok meteor ve rakip cesidi.

Orta Vadeli

- Guc-yukseltmeleri (kalkan, hizli ates, coklu atis).

- USB yerine Wi-Fi / ESP-NOW ile kablosuz kumandalar.
- Kalici yuksek-skor tablosu.

Uzun Vadeli

- Iki PC arasinda agdan oynama.
- Ozel PCB + 3B-baskili joystick muhafazasi.
- Oyun icinde ayarlanabilir kontrol / kalibrasyon ekrani.

HIZLI REFERANS KARTI

Bolunmus Ekran Uzay Oyunu -- ESP32-S3 + Cift Joystick

KONTROLLER

P1 hareket	W A S D	sol arena
P2 hareket	Yon tuslari	sag arena
Ates	OTOMATIK	iki oyuncu
Ozel guc	SPACE / ENTER	veya joystick butonu
Duraklat / Restart	P / R	ESC = cikis

PIN HARITASI (ESP32-S3)

P1 X / Y / SW	GPIO4 / 5 / 6	ADC1
P2 X / Y / SW	GPIO7 / 8 / 9	ADC1
Guc	3V3 + GND	iki joystick

CALISTIRMA

Oyun (klavye)	python main.py
Oyun (joystick)	python main.py --serial
ESP yukle	pio run -d firmware -t upload